Basic Programming Algorithms

Specification tool: *https://progalap.elte.hu/specifikacio/*)

Algorithm tool: *https://progalap.elte.hu/stuki/*)

1. Summation

General specification

In: b∈Z, e∈Z
Out: s∈H
Pre: -
Post: s=SUM(i=b..e, f(i))

Pattern algorithm

```
s:=0
i=b..e
  s:=s+f(i)
```

Specification for simpler task

In: n∈Z, m∈Z[1..n]
Out: s∈Z
Pre: -
Post: s=SUM(i=1..n, m[i])

Algorithm for simpler task

```
s:=0
i=1..n
  s:=s+m[i]
```

Specification for harder task

(Summarize income money excluding expenses)
In: n∈Z, m∈Money[1..n]
Money=(in:N x exp:N)
Out: s∈Z
Pre: -
Post: s=SUM(i=1..n, m[i].in-m[i].exp)

Algorithm for harder task

```
s:=0
i=1..n
  s:=s+m[i].in-m[i].exp
```

2. Counting

General specification

In: b∈Z, e∈Z
Out: cnt∈N
Pre: -
Post: cnt=COUNT(i=b..e, A(i))

Pattern algorithm

```
cnt:=0
i=b..e
  A(i)  T: cnt:=cnt+1  F: -
```

Specification for simpler task

(Count all the positive numbers (m[i]>0))
In: n∈Z, m∈Z[1..n]
Out: cnt∈N
Pre: -
Post: cnt=COUNT(i=1..n, m[i]>0)

Algorithm for simpler task

```
cnt:=0
i=1..n
  m[i]>0  T: cnt:=cnt+1  F: -
```

Specification for harder task

(Number of people born during winter)
In: n∈Z, m∈Z[1..n]
Out: cnt∈N
Pre: ∀i∈[1..n]:(1<=m[i]<=12)
Post: cnt=COUNT(i=1..n, (m[i]<3) or (m[i]=12))

Algorithm for harder task

```
cnt:=0
i=1..n
  (m[i]<3) or (m[i]=12)  T: cnt:=cnt+1  F: -
```

3. Maximum selection

General specification

In: b∈Z, e∈Z
Out: maxind∈Z, maxval∈H
Pre: b≤e
Post: maxind ∈ [b..e] and
∀i∈[b..e]:(f(maxind)≥f(i)) and
maxval=f(maxind)
shorting:
Post: (maxind, maxval)=MAX(i=b..e, f(i))

Pattern algorithm

```
maxval:=f(b)
maxind:=b
i=(b+1)..e
  f(i)>f(maxind)
  T: maxval:=f(i)
     maxind:=i
  F: -
```

Specification for simpler task

(Pick the tallest people)
In: n∈Z, m∈Z[1..n]
Out: maxind∈Z, maxval∈Z
Pre: n > 0
Post:(maxind, maxval) =
MAX(i=1..n, m[i])

Algorithm for simpler task

```
maxval:=m[1]
maxind:=1
i=2..n
  m[i]>m[maxind]
  T: maxval:=m[i]
     maxind:=i
  F: -
```

Specification for harder task

(Known: title and year of publications, pick the title of first newest one)
In: n∈Z, m∈Books[1..n]
Books=(book:S x year:Z)
Out: maxind∈Z, maxval∈Z, title∈S
Pre: n>0
Post: (maxind, maxval)=
MAX(i=1..n, m[i].year)
and title=m[maxind].book

Algorithm for harder task

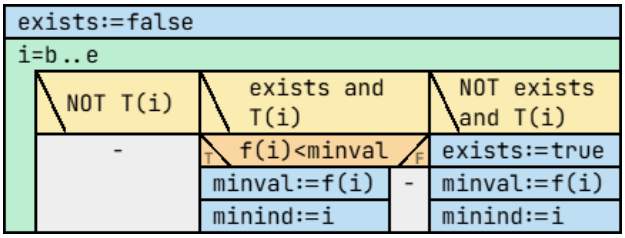
```
maxval:=m[1].year
maxind:=1
i=2..n
  m[i].year>m[maxind].year
  T: maxval:=m[i].year
     maxind:=i
  F: -
title:=m[i].book
```

4. Minimum selection with condition

General specification

In: b∈Z, e∈Z
Out: exists∈L, minind∈Z, minval∈H
Pre: b≤e
Post: (exists, minind, minval) =
MINIF(i = b..e, f(i), T(i))

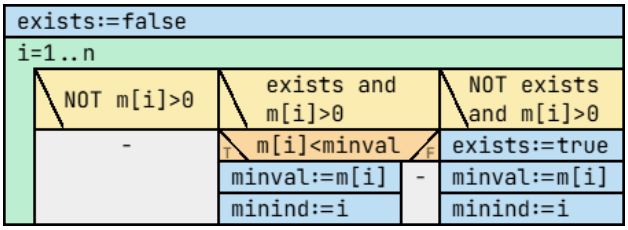
Pattern algorithm



Specification for simpler task

(Pick the smallest from positiv values)
In: n∈Z, m∈Z[1..n]
Out: maxind∈Z, minval∈Z
Pre: n > 0
Post:(exists, minind, minval) =
MINIF(i=1..n, m[i], m[i]>0)

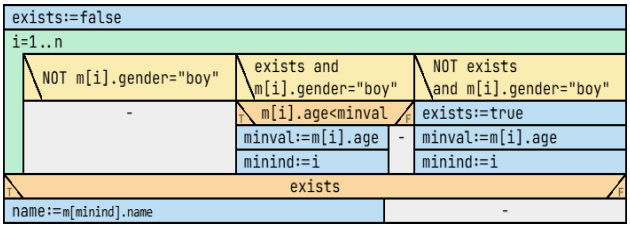
Algorithm for simpler task



Specification for harder task

(Known: name,gender and age of children, pick the name of youngest boy)
In: n∈Z, m∈Children[1..n],
Children=(name:S x gender:S x age:Z)
Out: exists∈L, minind∈Z, minval∈Z, name∈S
Pre: n > 0
Post: (exists, minind, minval)=
MINIF(i=1..n, m[i].age,
m[i].gender="boy") and
name=m[minind].name

Algorithm for harder task

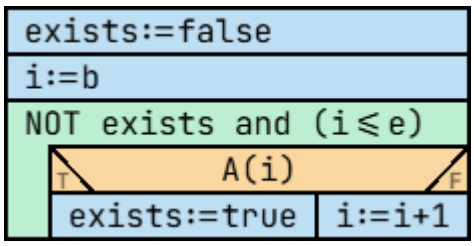


5. Decision

General specification

In: b∈Z, e∈Z
Out: exists∈L
Pre: -
Post: exists=(∃i∈[b..e]:(A(i))
shorting:
Post: exists=EXISTS(i=b..e, A(i))

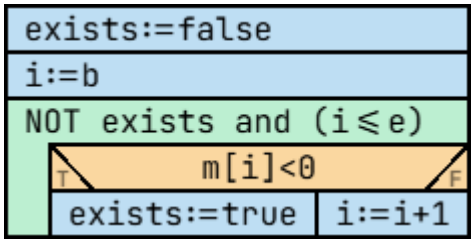
Pattern algorithm



Specification for simpler task

(Determine if array has negative element)
In: n∈Z, m∈Z[1..n]
Out: exists∈L
Pre: -
Post: exists=EXISTS(i=1..n, m[i]<0)

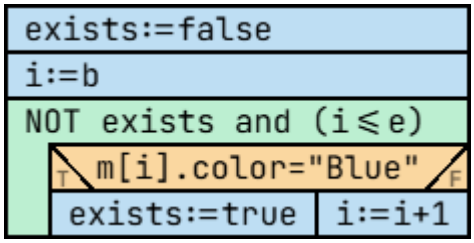
Algorithm for simpler task



Specification for harder task

(Known: quantity and color of candies, determine if there is a blue candy)
In: n∈Z, m∈Candies[1..n],
Candies=(quantity:Z x color:S)
Out: exists ∈ L
Pre: -
Post: exists=EXISTS(i=1..n,
m[i].color="Blue")

Algorithm for harder task



6. Decision all

General specification

In: b∈Z, e∈Z
Out: all∈L
Pre: -
Post: all=(∀i∈[b..e]:(A(i))
shorting:
Post: all=FORALL(i=b..e, A(i))

Specification for simpler task

(Determine all elements are negative)
In: n∈Z, m∈Z[1..n]
Out: all∈L
Pre: -
Post: all=FORALL(i=1..n, m[i]<0)

Specification for harder task

(Known: quantity and color of candies, determine all candy is blue)
In: n∈Z, m∈Candies[1..n],
Candies=(quantity:Z x color:S)
Out: all∈L
Pre: -
Post: all=FORALL(i=1..n,
m[i].color="Blue")

Pattern algorithm

```
i:=b
(i≤e) and A(i)
  i:=i+1
all:=(e<i)
```

Algorithm for simpler task

```
i:=1
(i≤n) and m[i]<0
  i:=i+1
all:=(n<i)
```

Algorithm for harder task

```
i:=1
(i≤n) and m[i].color="Blue"
  i:=i+1
all:=(n<i)
```

7. Selection

General specification

In: b∈Z, e∈Z
Out: ind∈Z
Pre: ∃i∈[b..e]:(T(i))
Post: ind>=b and T(ind) and
∀i∈[b..(ind-1)]: NOT(T(i))
shorting:
Post: ind=SELECT(i≥b, T(i))

Pattern algorithm

```
ind:=b
NOT (A(ind)
  ind:=ind+1
```

Specification for simpler task

(Select the first element's index equals to 10)
In: *n∈N, m∈N[1..n]*
Out: *ind∈N*
Pre: *n ≥ 1 and ∃i∈[1..n]:(m[i]=10)*
Post: *ind = SELECT(i>=1, m[i] = 10)*
and ind≤n

Algorithm for simpler task

```
ind:=1
NOT (m[ind]=10
  ind:=ind+1
```

Specification for medium task

(Binary digits are 0 or 1. Known a sequence of one-digit numbers. Select the one which is NOT a binary digit!)
In: n ∈ N, m∈N[1..n]
Out: ind ∈ N
Pre: n >= 1 and ∃i∈[1..n]:
((m[i] != 1) and (m[i] != 0))
Post: ind = SELECT(i>=1, (m[i] != 1)
and (m[i] != 0)) and 1 ≤ ind ≤ n

Algorithm for medium task

```
ind:=1
(m[ind]≠0) and (m[ind]≠1)
  ind:=ind+1
```

Specification for harder task

(Known: the size and power of flashlights. Select the one which has a power of at least 3, and is medium-sized or smaller!)
In: n ∈ N, m∈Flashlights[1..n]
Flashlights=(size:S x power:N)
Out: ind ∈ N
Pre: n ≥ 1 and ∃i∈[1..n]:(m[i].power>=3)
and ((m[i].size="Small") or
(m[i].size="Medium"))
Post: ind=SELECT(ind>=1,((m[ind].power>=3)
and ((m[ind].size="Small") or
(m[ind].size="Medium")))) and
1 ≤ ind ≤ n

Algorithm for harder task

```
ind := 1
((m[ind].power ⩾ 3) and
((m[ind].size="Small") or
   (m[ind].size="Medium")))
  ind := ind + 1
```

8. Search

General specification

In: b∈Z, e∈Z
Out: exists∈L, ind∈Z
Pre: -
Post: exists=∃i∈[b..e]:(A(i)) and
exists->(ind∈[b..e] and A(ind)
and
∀i∈[b..(ind-1)]:(not A(i)))
shorting:
Post: (exists,ind)=SEARCH(i=b..e,A(i))

Pattern algorithm

```
i:=b
(i≤e) and (not A(i))
  i:=i+1
exists:=(i≤e)
T        exists        F
ind:=i        -
```

Specification for simpler task

(Some numbers in array, search for the number 5. We do NOT know it is exist or not in an array.)
In: n∈Z, m∈Z[1..n]
Out: exists∈L, ind∈Z
Pre: -
Post: (exists, ind)=SEARCH(i=1..n,
m[i]=5)

Algorithm for simpler task

```
i:=1
(i≤n) and (m[i]≠5)
  i:=i+1
exists:=(i≤n)
T        exists        F
ind:=i        -
```

Specification for medium task

An array contains animals, search for the one that is NOT a mammal)
In: n ∈ Z, m ∈ Animals[1..n],
Animals=(race:S x mammal:L)
Out: exists ∈ L, ind ∈ Z
Pre: -
Post: (exists, ind)=SEARCH(i=1..n,
(not m[i].mammal))

Algorithm for medium task

```
i:=1
(i≤n) and NOT(m[i].mammmal)
  i:=i+1
exists:=(i≤n)
T        exists        F
ind:=i        -
```

Specification for harder task

(Known: pets's name, legs number, mammal
Search the first mammal with 4 legs if it exists)
In: n ∈ Z, m ∈ Animals[1..n],
Animals=(name:Sxlegs:Zxmammal:L)
Out: exists ∈ L, ind ∈ Z
Pre: -
Post: (exists, ind)=SEARCH(i=1..n,
(m[i].mammal and
(m[i].legs=4)))

Algorithm for harder task

```
i:=1
(i≤n) and
not(m[i].mammal and (m[i].legs=4))
  i:=i+1
exists:=(i≤n)
T        exists        F
ind:=i        -
```

9. Copy

General specification

In: b∈Z, e∈Z
Out: y∈H[1..e-b+1]
Pre: -
Post: ∀i∈[b..e]:(y[i-b+1]=f(i))
shorting:
Post: y=COPY(i=b..e, f(i))

Pattern algorithm

```
i=b..e
  y[i-b+1]:=f(i)
```

Specification for simpler task

(Store doubled value)
In: n∈Z, m∈Z[1..n]
Out: r∈Z[1..n]
Pre: -
Post: r=COPY(i=1..n, m[i]*2)

Algorithm for simpler task

```
i=1..n
  r[i]:=m[i]*2
```

Specification for harder task

(the numbers from matrix's diagonal)
In: n ∈ Z, m ∈ Z[1..n, 1..n]
Out: r ∈ Z[1..n]
Pre: -
Post: r=COPY(i=1..n, m[i,i])

Algorithm for harder task

```
i=1..n
  r[i]:=m[i,i]
```

10. Multiple item selection

General specification

In: b∈Z, e∈Z
Out: cnt∈N, y∈H[1..cnt]
Pre: -
Post: (cnt, y)=MIS(i=b..e, A(i), f(i))

Pattern algorithm

```
cnt:=0
i=b..e
    A(i)
    T: cnt:=cnt+1
       y[cnt]:=f(i)
    F: -
```

Specification for simpler task

(Store doubled value)
In: n ∈ Z, m∈Z[1..n]
Out: cnt∈N, y∈Z[1..cnt]
Pre: -
Post: (cnt, y)=MIS(i=1..n,(m[i]%2=0), m[i])

Algorithm for simpler task

```
cnt:=0
i=1..n
    (m[i]%2)=0
    T: cnt:=cnt+1
       y[cnt]:=m[i]
    F: -
```

Specification for medium task

(balls of different colors and weights; Select the weights of red balls)
In: n ∈ Z, m ∈ Balls[1..n],
Balls=(color:S x weight:R)
Out: cnt ∈ N, y ∈ R[1..cnt]
Pre: -
Post: (cnt, y) = MIS(i=1..n, m[i].color = "Red", m[i].weight)

Algorithm for medium task

```
cnt:=0
i=1..n
    m[i].color="Red"
    T: cnt:=cnt+1
       y[cnt]:=m[i].weight
    F: -
```

Specification for harder task

(We have two arrays: one with car colors, and another with the same car's type and brand.
Select the car's brand that are blue and not fully electric.)
In: n ∈ Z, carColors ∈ S[1..n],
carTypes∈CarTypesAndBrands[1..n],
CarTypesAndBrands=(type:Sxbrand:S)
Out: cnt ∈ N, y ∈ S[1..cnt]
Pre: -
Post: (cnt, y) = MIS(i=1..n, carColors[i] = "Blue" and carTypes[i].type != "Electric", carTypes[i].brand)

Algorithm for harder task

```
cnt:=0
i=1..n
    (carColors[i]="Blue") and
    (carTypes[i].type≠Electric)
    T: cnt:=cnt+1
       y[cnt]:=carTypes[i].brand
    F: -
```